

A small decorative horizontal bar with a teal segment on the left and an orange segment on the right.

CSC 84040: Advanced Data Mining *with AI Agents*

Lecture 02: Distributed Techniques for Data Mining, MapReduce, Frequent Item Sets, and more

By Saptarashmi Bandyopadhyay

Email: sbandyopadhyay@ccny.cuny.edu

Assistant Professor of Computer Science

City College of New York and Graduate Center at the City University of New York

September 9, 2026 CSC 84040

Distributed Processing for Autonomous Data Mining

—



Why Distributed?

In order to train a well-behaved agent (e.g. AlphaGo, StarCraft bot, etc.) we need to train on way more interactions (billions+) than we could feasibly do in a single thread or even a single GPU.

The Math:

- Time per Step (t_{step}) $\times$ Required Steps (N_{steps}) = Total Training Time.
- If $t_{step} = 0.1s$ and $N_{steps} = 10^9 \rightarrow 3$ years to train.

The Solution: Distributed Processing. We cannot reduce N_{steps} easily, so we must perform steps simultaneously.



The Two Main Architectures

1. Data Parallelism (The "Rollout" Worker):

- *Concept:* Copy the Agent to 100 CPUs. Each Agent plays the game independently. They send their data to a central "Learner" to update the brain.
- *Relevance:* This is how standard RL (PPO/A3C) works.

2. Model Parallelism (The "Giant Brain"):

- *Concept:* The Agent is too big for one GPU (e.g., a 70B parameter LLM). We split the *layers* across multiple GPUs.
- *Relevance:* Essential for "LLM Agents."

Data Parallelism

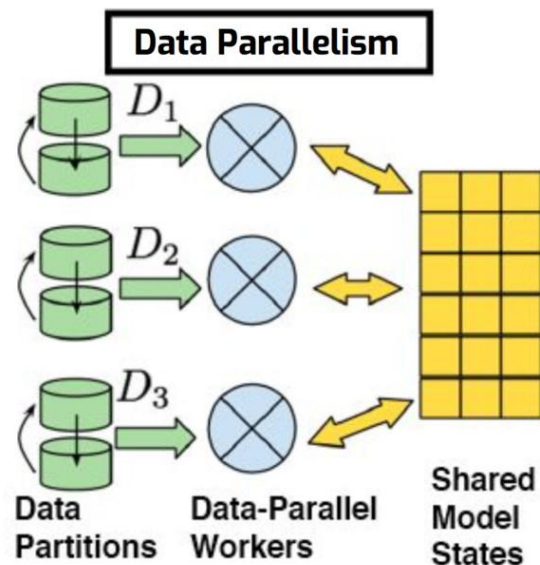
Copies of the same Agent across N devices (CPUs or GPUs).

How it Works:

- Each worker runs its own instance of the environment.
- They collect experience (data) independently and send it to a central "Learner" to update the shared brain.

Why we use it:

- **Throughput:** Increases experience collection linearly with N .
- **Relevance:** The standard approach for PPO, A3C, and most projects in this class (e.g., using JAX `vmap`).



Model Parallelism

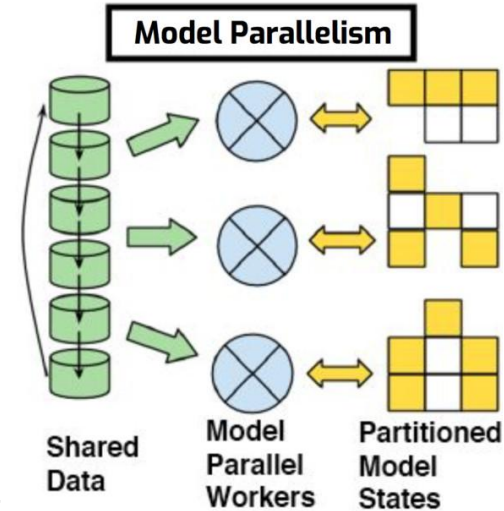
Split a **single** giant Agent across multiple GPUs.

How it Works:

- The model (e.g., 70B+ parameters) is too big for one GPU
- The data flows end to end through a model distributed across multiple GPUs (e.g. layers 1-10 in GPU 1, layers 11-20 in GPU 2)

Why we use it:

- **Capacity:** Essential for "LLM Agents" and Foundation Models that physically cannot fit on one device.
- **Trade-off:** High communication overhead (latency) between GPUs.





Synchronous vs. Asynchronous (A3C vs. PPO)

Synchronous (PPO): Everyone waits for the slowest worker. "Stop, update weights, resume." This tends to be more stable, theoretically speaking, but slows the whole system down.

Asynchronous (A3C/Impala): Workers don't wait. They dump data and keep going. (Faster, but mathematically "messy" due to lag).

Your Projects: You will likely use **Synchronous** methods (via JAX/Gymnasium) because they are easier to debug.



Synchronous Distributed Agents

Workers act like an orchestra. They collect data together and pause together.

Mechanism (e.g., PPO, A2C):

- **Step 1:** All workers collect N steps of experience.
- **Step 2:** They **pause** and send data to the Learner.
- **Step 3:** The Learner updates the model and sends new weights to *all* workers.
- **Step 4:** Workers resume.

Pros & Cons:

- **Stable:** Mathematically cleaner; everyone uses the same policy version.
- **Debuggable:** Easier to reproduce bugs.
- **The "Straggler" Problem:** The whole system waits for the slowest worker.



Asynchronous Distributed Agents

Jazz musicians playing at their own tempo, updating the central brain whenever they are ready.

Mechanism (e.g., A3C, IMPALA):

- Workers run independently without waiting for others.
- When a worker finishes a batch, it pushes gradients to the **Parameter Server** immediately and pulls the latest weights.

Pros & Cons:

- **Fast:** No waiting for slow workers; maximum hardware utilization.
- **Stale Gradients:** Worker may calculate gradients using an old model, destabilizing learning.
- **✗ Chaos:** Hard to debug; runs are non-deterministic.

Modalities (Types of Data)

—



Modality 1 - Tabular/Vector Data

Definition: The state is a list of numbers. In a Gridworld, this might be (x, y) coordinates, in a stock market simulation, this might be (price, volume) of the market.

Pros: Fast, low memory, interpretable.

Cons: Lose spatial/relational information. A list of pixels is not the same as an image.

TL;DR: Good for simple control tasks, bad for the "Real World."



Modality 2 - Graphical Data (GNNs)

Much of the time, we can't treat data as isolated data points; they *depend* and *relate*.

We can model a Supply Chain as a graph where different nodes directly depend on each other

- Factory A $\rightarrow$ Truck B $\rightarrow$ Store C.
- If Truck B breaks, it affects A and C directly.

Graph Neural Networks (GNNs):

- Agents process "Nodes" and "Edges."
- Allows the agent to understand *relationships* and *topology* (who is connected to whom?).

Research Hotspot: Multi-Agent Communication is a graph problem!



Modality 3 - Multi-Modal

Definition: Combining disparate data streams.

Visual-Language Models (VLM): "Red light" (Image) + "Stop sign text" (Language) $\rightarrow$ Action: Brake.

Audio-Visual: Hearing a siren + Seeing the ambulance.

The Challenge: Alignment. How do you map the word "Dog" (Text embedding) to a picture of a Dog (Pixel embedding)?

If you are building a "Robotic Agent" or "AI Agent Assistant," you *must* handle multi-modality.

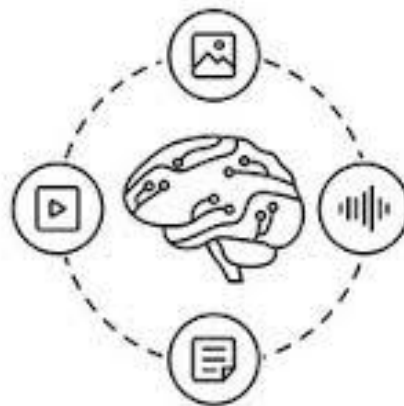
What is Multimodality?

So far, many of the problems we've looked at have focused on a single particular modality (data type).

This could be text in LLMs, images in a object detection model, or sensors in a home IoT device such as a thermostat.

Multimodality is a property given to models that deal with more than one type of data. For example:

- Autonomous Vehicles have video, sound, LiDAR, radar, etc.
- Smart Glasses have video, images, sound, etc.





Examples of Modalities

There are many, many kinds of data that AI models can use, including but not limited to:

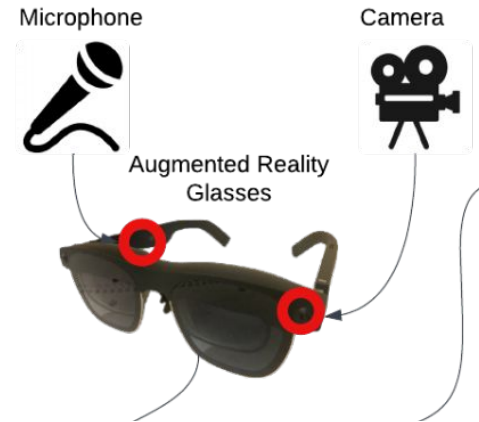
- **Language:** Written text and/or spoken audio commands
- **Visual:** Images and/or dynamic video feeds
- **Audio:** Speech, environmental noises, music, etc.
- **Sensors:** LiDAR, radar, IMU (inertial measurement units), etc.

By combining two or more of these modalities, we can move AI towards a more human-like perception of the world.

Real-World Use Case: Personal AI Assistants

Personal AI Assistants often exist on a user's personal device ("edge device") such as a smartphone, smart glasses, smart watch, etc.

A prominent example is Google Deepmind's Astra Augmented Reality Glasses or Meta's Smart Ray-Ban Glasses



Real-World Use Case: Autonomous Vehicles

Self-driving cars are highly multimodal. In a Waymo car we have:

- High-resolution cameras
- LiDAR (Light Detection and Ranging) Sensors
- Radar
- Microphones

Tesla cars are known as “vision-only” but even they combine cameras microphones, map data, and sensors in their end-to-end decision-making.





Connection to Data Mining

Real-world data is extremely complex, far from the clean tables and text used by traditional data mining techniques. Integrating multiple formats transforms data mining in several key ways:

- Combining user reviews (text) with shared photos (images) on social media gives a complete picture of customer sentiment rather than looking at words alone.
- Fusing diverse inputs improves classification, extraction, and predictive performance across complex domains.
- Information from a data-rich modality (like text) can help train and interpret models for a data-poor modality (like specialized sensor logs or rare medical images)

MapReduce

—

Saptarashmi Bandyopadhyay

Working with Big Data

In modern Distributed Computing and AI, we're often working with internet-scale datasets. How do we process so much data?

Traditional Approach (Serial Processing):

- Store data on one massive storage server (SAN/NAS).
- Pull data across the network to a supercomputer.
- Process line-by-line.

The Bottlenecks:

- Moving 10PB of data takes an extremely long time!
- We can't afford to start over if we crash at 99% downloaded
- Linear reads from disk I/O is also very slow





Divide and Conquer

Google addressed this problem back in 2004:

- Don't buy one supercomputer. *Buy 1,000 cheap commodity servers.*
- Data Locality: Don't move data to the CPU. *Move the code to the data.*

The Abstraction:

- Hide the messy details (network failures, disk crashes, load balancing).
- Expose two simple functions to the programmer: Map and Reduce.

The MapReduce Deli Shop

Goal: Make 1,000 Sandwiches as fast as possible.

Phase 1: MAP (The Prep Cooks)

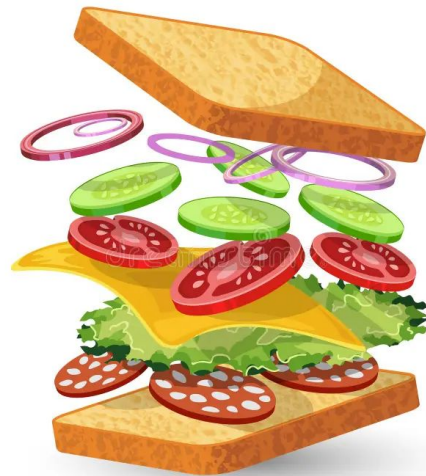
- 5 cooks working in parallel.
- They don't make full sandwiches. They just output intermediate parts.
- Output: Piles of "Sliced Tomato," "Chopped Lettuce," "Cooked Bacon."

Phase 2: SHUFFLE (The Runners)

- The messy part. Runners grab all tomatoes from all 5 cooks and put them in the "Tomato Bin."
- Grouping: Ensure all instances of the same ingredient end up in the same place.

Phase 3: REDUCE (The Assemblers)

- One chef stands at the Tomato bin, one at the Bacon bin.
- They process the piles into the final result.



The Technical Approach

Input: Split into chunks (usually 64MB or 128MB).

MAP: (Key, Value) -> List(Key, Value)

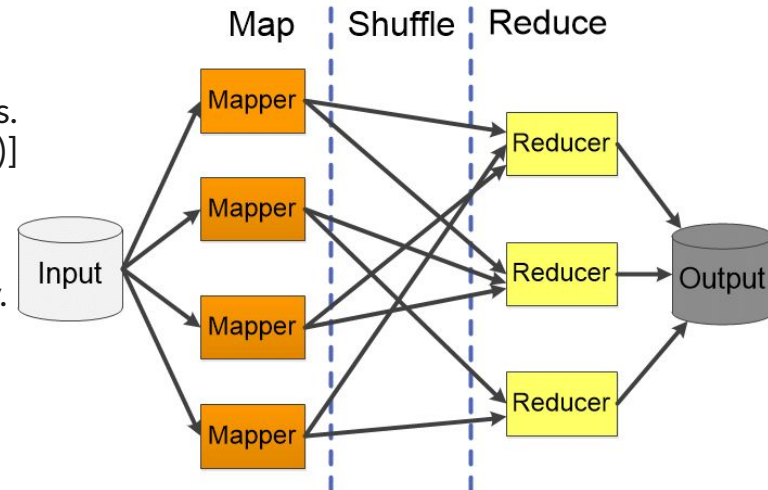
- Takes raw data, filters/parses it, outputs intermediate pairs.
- Example: (Doc1, "Hello World") -> [("Hello", 1), ("World", 1)]

SHUFFLE & SORT: (Key, Value) -> (Key, List[Values])

- The system groups all values associated with the same key.
- Example: ("Hello", [1, 1, 1, 1])

REDUCE: (Key, List[Values]) -> (Key, Result)

- Aggregates the list into a single output.
- Example: ("Hello", 4)





Example: Word Count

Input Data: "Deer Bear River" "Car Car River" "Deer Car Bear"

Map Phase (Parallel):

- Worker 1: (Deer, 1), (Bear, 1), (River, 1)
- Worker 2: (Car, 1), (Car, 1), (River, 1)
- Worker 3: (Deer, 1), (Car, 1), (Bear, 1)

Shuffle Phase (Network Intensive): Moves data so keys group together.

- Bear: [1, 1]
- Car: [1, 1, 1]
- Deer: [1, 1]
- River: [1, 1]

Reduce Phase: Sum the lists: Bear: 2, Car: 3, Deer: 2, River: 2



Overhead

Synchronization Barrier

- The Reduce phase cannot start until the slowest Map worker is finished.
- Analogy: You can't count total "Tomatoes" until the last prep cook finishes chopping.

Network Congestion:

- The "Shuffle" phase requires moving massive amounts of data between servers simultaneously.

Fault Tolerance:

- What if Worker 5 dies? The Master node must detect it and re-assign that chunk of data to Worker 2.



Hands-On MapReduce Demonstration

[https://colab.research.google.com/drive/1X7Bvwi5VIUblas5OeeltXaTyXHYGAdPf?usp=drive link](https://colab.research.google.com/drive/1X7Bvwi5VIUblas5OeeltXaTyXHYGAdPf?usp=drive_link)

—

Saptarashmi Bandyopadhyay



Data Quality and Searching for Anomalies

- The objective of data mining is often an algorithmic approach rather than merely fitting a statistical model. Today, now more than ever, we need to be able to handle massive large-scale data for getting the best and most accurate results.
- A common data mining task involves discovering unusual events hidden within massive datasets. Without proper theoretical boundaries, searching broadly for suspicious or unusual activities will yield bogus results. These bogus occurrences are statistical artifacts that occur strictly because random data will inevitably contain features that appear significant.



Bonferroni's Principle

“If you look in your data for too many things at the same time, you will see things that look interesting, but are in fact simply statistical artifacts and have no significance.”

The Market-Basket Model

The dataset consists of a large collection of "baskets," where each basket contains a small set of "items".

The fundamental goal is to find small sets of items that frequently appear together across many baskets.

Let s be a support threshold. An itemset I is frequent if it appears in at least s baskets.

We can get the strongest dependencies between items this way.

Market Basket Analysis





Association Rules

Association rules extract predictive relationships from frequent itemsets, denoted mathematically as $I \rightarrow j$, where I is a set of items and j is a single item.

The rule implies that if a basket contains all items in I , it is statistically likely to contain item j .

The confidence c of an association rule $I \rightarrow j$ is defined mathematically as the ratio of the support for $I \cup \{j\}$ to the support for I .

Finding high-confidence rules requires first identifying all frequent itemsets $I \cup \{j\}$ and their subsets I .

We need to consider monotonicity as well. If a set I is frequent, its subset $J \subseteq I$ must also be frequent.

Conversely, if an itemset is infrequent, all of its supersets must strictly be infrequent.

The Apriori Algorithm

—



Introduction

The A-Priori Algorithm exploits the monotonicity property of itemsets to constrain main memory usage.

It only generates candidate itemsets of size k if all their subsets of size $k - 1$ are previously proven to be frequent.



Apriori: First Pass

The first algorithm pass scans the dataset to count the specific occurrences of each individual item.

A translation table is often built in memory to map item names to integer indices from 1 to n .

An array is used to count the occurrences, identifying which individual items satisfy the predefined support threshold s .

Items that meet or exceed the support threshold s are verified as frequent items.

A new numbering system is generated, assigning contiguous integers from 1 to m exclusively for the frequent items, which vastly compresses the required memory footprint.



Second Pass

The second pass focuses solely on candidate pairs formed by the subset of frequent items identified in the first pass.

It scans the baskets again, evaluating possible pairs for each basket only if both items in the pair are individually frequent.

Occurrences of these filtered candidate pairs are aggregated to find the true frequent pairs.



Apriori: Generalization to Size k

The algorithm creates a set of candidate itemsets C_k and scans the baskets to count their support. Any itemsets failing to satisfy the support threshold are pruned before iterating to $k + 1$.



Limitations of Apriori

Apriori fundamentally requires generating all candidate combinations of items.

It requires multiple full scans of the transactional database, which degrades computational speed on massive datasets.

FP-Growth

—



Introduction

The FP-Growth (Frequent Pattern Growth) algorithm mines frequent itemsets without engaging in costly candidate generation.

It compresses the dataset into a highly compact graph-like structure known as the Frequent Pattern Tree (FP-Tree).



FP-Growth Item Frequency and Sorting

The algorithm first scans the entire dataset to compute the exact frequencies of individual items.

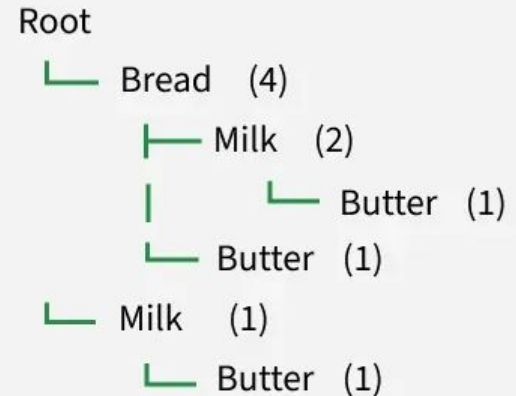
It filters out infrequent items and sorts the items in every individual transaction in descending order based on their overall frequency in the dataset.

Transaction ID	Sorted Items by Frequency
T1	Bread, Milk, Butter
T2	Bread , Butter
T3	Bread, Milk
T4	Milk , Butter
T5	Bread

Constructing the FP-Tree

The sorted transactions are inserted iteratively into the FP-tree structure.

During insertion, shared prefixes among different transactions are merged, which compactly stores the data and drastically minimizes redundancy.





Conditional Pattern Bases

After building the main FP-tree, the algorithm determines a "conditional pattern base" for each item.

The conditional pattern base consists of the specific prefix paths within the FP-tree that co-occur with the target item.

[(Bread, Milk): 1, (Bread): 1, (Milk): 1]



Constructing Conditional FP-Trees

FP-Growth will build a more compact, local FP-tree for every individual item by way of the conditional pattern base it has extracted.

In so doing, the approach confines the search space and is well suited to the efficient computation of any frequent patterns associated with that item.

Since both meet the support threshold (≥ 2), we can now generate frequent patterns:

- {Butter, Bread}; {Butter, Milk}; {Butter, Bread, Milk}

(Bread, Milk): 1

(Bread): 1

(Milk): 1

Count all items:

Bread: 2

Milk: 2

Mining the Tree Recursively



Pattern Extraction

The algorithm examines these localized conditional trees to extract patterns that independently satisfy the minimum support threshold.



Recursive Breakdown

This approach recursively breaks down the primary tree into smaller structures, mining deeper itemsets efficiently.



Efficiency Advantages of FP-Growth

With the FP-Growth algorithm, a database is scanned no more than twice in strict fashion, the maximum pattern length being of no consequence.

This approach sidesteps the candidate generation phase altogether and in doing so overcomes the kind of exponential search space bottleneck that is characteristic of Apriori.

The result is an algorithm of considerable scale that performs with distinction on large, dense datasets.



Limitations of FP-Growth

With a highly varied dataset, the FP-tree will grow in size and the memory it takes to store can be considerable.

In terms of construction and interpretation, one will find it a much more arduous task than with the Apriori algorithm, owing to the intricacies of the recursive logic and tree implementation.

Questions?

—